

AhamX Bodhi

Where I Becomes Infinite

Multi-Head Attention

Module 2.2: Transformer Architecture Fundamentals
Prof. Sashikumaar Ganesan | IISc Bengaluru



Session Overview

What You Will Learn

- Why one attention head is not enough
- How multiple heads run in parallel
- The concatenate-and-project output step
- Head specialisation: what each head learns
- Modern variants: GQA and MQA

Concept at a Glance

- **Duration:** 10 minutes
- **Bloom's Level:** B2 – Understand
- **Difficulty:** L2 – Intermediate-Beginner
- **Module:** 2.2 Transformer Architecture Fundamentals
- **Prerequisite:** Self-Attention Mechanism



Learning Objectives

By the End of This Session You Will Be Able To

1. **Explain** why multi-head attention offers richer representations than a single head
2. **Describe** how h attention heads are computed in parallel from a shared input
3. **Trace** the full Multi-Head Attention (MHA) forward pass: split, attend, concatenate, project
4. **Interpret** the role of the output projection matrix $\mathbf{W}^O$
5. **Distinguish** Multi-Head Attention from Grouped-Query Attention (GQA) and Multi-Query Attention (MQA)



Prerequisites and Connections

You Should Already Know

- Scaled dot-product attention formula
- Q, K, V projection matrices and their roles
- Softmax as a probability distribution
- Matrix multiplication basics

This Concept Unlocks

- Positional Encoding (Module 2.4)
- Feed-Forward Networks in Transformers (Module 2.5)
- Decoder-Only Architecture (Module 2.6)
- Cross-Attention in encoder-decoder models
- LoRA: Low-Rank Adaptation (Module 7)
- Flash Attention and inference optimisation (Module 9)



The Limitation of a Single Attention Head

One Head, One Perspective

- A single attention head computes one set of attention weights — one view of which tokens are relevant to each other
- Natural language has **multiple simultaneous relationships**: syntactic structure, semantic similarity, coreference, positional proximity
- A single head must try to capture all these at once within its fixed d_k -dimensional projection subspace
- This creates an information bottleneck: complex, multi-faceted dependencies are compressed into a single scalar attention score per token pair

The solution is to run **multiple attention heads in parallel**, each learning a different projection of the same input — and then merging their perspectives.



The Core Idea --- Multiple Parallel Perspectives

What Each Head Can Specialise In

- **Head 1 (syntactic):** Attends to grammatical dependencies — subject to verb, noun to its modifier
- **Head 2 (coreference):** Links pronouns to their antecedents ("she" → "Alice")
- **Head 3 (positional):** Attends to nearby tokens regardless of content
- **Head 4 (semantic):** Connects semantically related tokens across long distances
- **Head 5 (task-specific):** Learns patterns driven by the fine-tuning objective

These specialisations **emerge from training** — they are not manually programmed. Different heads learn different projections from the same token embeddings.



The Multi-Head Attention Formula

Formal Definition

For h heads and input $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{model}}}$:

$$\text{head}_i = \text{Attention}(\mathbf{X}\mathbf{W}_i^Q, \mathbf{X}\mathbf{W}_i^K, \mathbf{X}\mathbf{W}_i^V)$$

$$\text{MultiHead}(\mathbf{X}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O$$

where each $\mathbf{W}_i^Q, \mathbf{W}_i^K, \mathbf{W}_i^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $d_k = d_{\text{model}}/h$, $\mathbf{W}^O \in \mathbb{R}^{h d_k \times d_{\text{model}}}$.

Key Insight: Dimension Preservation

Each head works in a **reduced** d_k -dimensional space. Concatenating h heads recovers $h \cdot d_k = d_{\text{model}}$ dimensions, so the output projection $\mathbf{W}^O$ maps back to the original model dimension. Total compute is roughly equal to one full-dimension single-head attention.

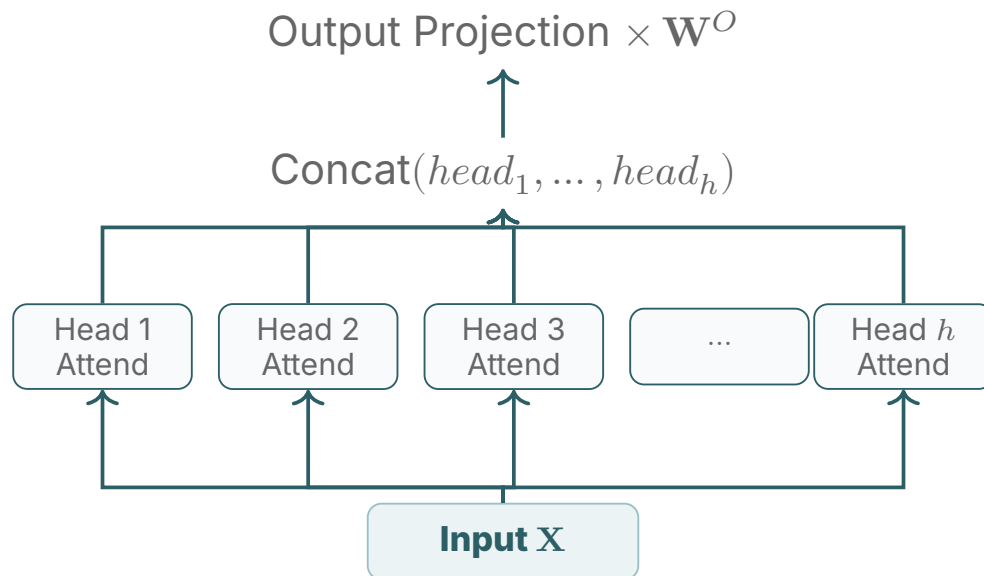


MHA Forward Pass --- Four Steps

Tracing a Single Forward Pass Through MHA

1. **Project:** Multiply input $\mathbf{X}$ by h distinct sets of $\mathbf{W}_i^Q, \mathbf{W}_i^K, \mathbf{W}_i^V$ to produce h sets of Q, K, V matrices (each of dimension d_k)
2. **Attend:** Compute scaled dot-product attention independently for each of the h heads — all heads run in parallel on GPU
3. **Concatenate:** Stack the h output matrices along the feature dimension to form a tensor of shape $\mathbb{R}^{n \times h d_k} = \mathbb{R}^{n \times d_{\text{model}}}$
4. **Project:** Multiply by the output weight matrix $\mathbf{W}^O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ to produce the final MHA output of shape $\mathbb{R}^{n \times d_{\text{model}}}$

Multi-Head Attention --- Architecture Diagram





Dimension Walkthrough --- GPT-2 Small as Example

GPT-2 Small: $d_{\text{model}} = 768$, $h = 12$ heads

- Each head dimension: $d_k = 768/12 = 64$
- Per head: $\mathbf{W}_i^Q, \mathbf{W}_i^K, \mathbf{W}_i^V \in \mathbb{R}^{768 \times 64}$ — each head gets its own learned subspace
- Per-head attention output: $\mathbb{R}^{n \times 64}$ for sequence of n tokens
- After concatenation: $\mathbb{R}^{n \times (12 \times 64)} = \mathbb{R}^{n \times 768}$
- After output projection $\mathbf{W}^O \in \mathbb{R}^{768 \times 768}$: final output $\mathbb{R}^{n \times 768}$ — same shape as input
- Learnable parameters per MHA layer: $4 \times d_{\text{model}}^2 = 4 \times 768^2 \approx 2.36\text{M}$



Head Counts in Production Models

Model	d_{model}	Heads (h)	d_k	Attention Type
GPT-2 Small	768	12	64	MHA
GPT-2 XL	1600	25	64	MHA
GPT-3 (175B)	12288	96	128	MHA
Llama 2 7B	4096	32	128	GQA (32Q / 8KV)
Llama 3 8B	4096	32	128	GQA (32Q / 8KV)
Llama 3.1 405B	16384	128	128	GQA
Mistral 7B	4096	32	128	GQA (32Q / 8KV)



Why MHA Does Not Cost h Times More

Compute Cost Analysis

- Each head uses $d_k = d_{\text{model}}/h$ — dimension shrinks proportionally
- The total projection size across all heads is still d_{model}
- Q, K, V projections: $3 \times n \times d_{\text{model}} \times d_k \times h = 3 \times n \times d_{\text{model}}^2$ — same as single full-dim head
- Attention per head: $O(n^2 d_k)$; total: $O(n^2 d_k \cdot h) = O(n^2 d_{\text{model}})$
- Output projection: $n \times d_{\text{model}} \times d_{\text{model}}$ — same cost

Multi-head attention achieves richer representations at the **same asymptotic cost** as a single head of full dimension — the dimensionality reduction per head is the key.



Modern Variants --- MQA and GQA

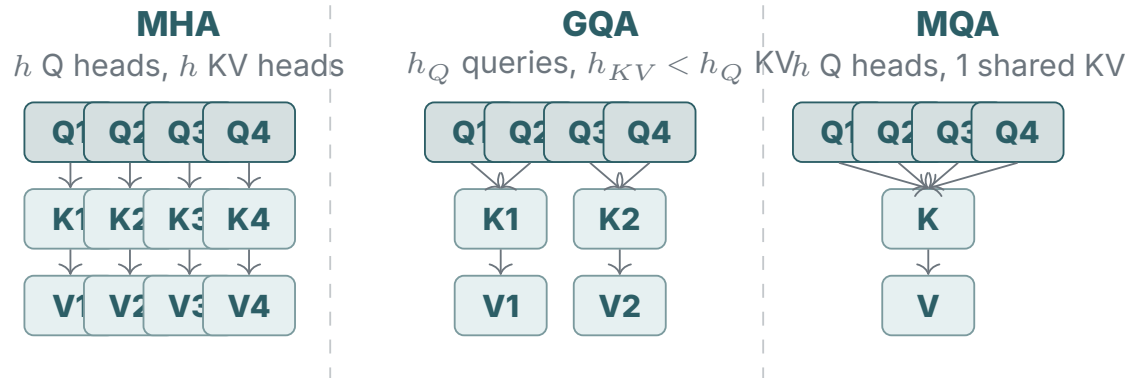
The KV-Cache Bottleneck Motivates New Designs

- In standard MHA, each of h heads stores its own $\mathbf{K}$ and $\mathbf{V}$ in the KV cache
- For Llama 3 70B with 128K context, MHA KV cache would exceed 640 GB
- **Multi-Query Attention (MQA)**: All query heads share a single $\mathbf{K}$, $\mathbf{V}$ head — dramatically smaller KV cache, some quality loss
- **Grouped-Query Attention (GQA)**: h_Q query heads share h_{KV} $\mathbf{K}/\mathbf{V}$ heads where $h_{KV} < h_Q$ — a quality-efficiency trade-off between MHA and MQA

GQA in Production: Llama 2 / 3, Mistral, Gemma

- Llama 3 8B: 32 query heads, 8 KV heads (4:1 ratio) — $4\times$ KV cache reduction
- GQA is now the default choice for efficient large-scale LLM deployment
- Quality nearly matches MHA at a fraction of the inference memory cost

MHA vs. GQA vs. MQA --- Visual Comparison





Cross-Attention --- When Q and K/V Come from Different Sources

Self-Attention Recap

- Q, K, V all derived from the **same** input sequence X
- Used in encoder blocks (BERT) and decoder blocks (GPT)
- Captures relationships *within* a single sequence

Cross-Attention

- Q from decoder sequence; K and V from encoder output
- Used in encoder-decoder models (T5, original Transformer)
- Also used in diffusion models: Q from noisy image, K/V from text
- Captures relationships *across* two different sequences



GenAI Application 1 --- Conversational AI

How Multi-Head Attention Powers ChatGPT, Claude, Gemini

- GPT-4 uses 128 attention heads per layer across 128 Transformer layers
- Each forward pass for a reply token attends over the entire conversation history using all 128 heads simultaneously
- **Claude's 200K-token context window:** requires highly efficient attention across all 200K positions for every generated token
- Different heads handle discourse coherence, entity tracking, instruction following, and factual grounding in parallel
- The output projection $\mathbf{W}^O$ fuses all heads into the residual stream that feeds the next Transformer block

Multi-head attention is the mechanism by which these models “read” the entire conversation before writing each new word.



GenAI Application 2 --- Code Generation

How Multi-Head Attention Enables GitHub Copilot

- **Syntactic heads:** Track bracket matching, indentation levels, and code block structure
- **Semantic heads:** Connect variable names to their declarations across hundreds of lines
- **Cross-file heads:** In multi-file context models, attend to imported functions and class definitions from other files
- **Task heads:** Focus on the function signature and docstring to guide the completion toward the correct behaviour
- **Pattern heads:** Recognise recurring idioms (e.g., try-except, list comprehensions, API call patterns)

CodeLlama (34B) uses 48 attention heads; StarCoder uses 48 heads — all of these co-activate on every token to produce contextually correct code.



GenAI Application 3 --- Multimodal Models

Cross-Attention in Image-Language Models

- **DALL-E 3 / Stable Diffusion XL:** Cross-attention layers allow image patches (Q) to attend to text token embeddings (K, V)
- At each denoising step, every image region attends to the full text prompt — spatial layout, colours, objects emerge from these multi-head cross-attention maps
- **GPT-4V / Claude 3 Vision:** Image patch tokens and text tokens are processed jointly through shared self-attention layers
- **Flamingo (DeepMind):** Gated cross-attention layers interleave visual features into a frozen language model via MHA

Multi-Head is Critical for Grounding

Multiple heads let the model simultaneously track spatial position, object identity, colour, and semantic alignment between image regions and text tokens.



GenAI Application 4 --- RAG and Long-Context Retrieval

Multi-Head Attention Across Retrieved Documents

- In a RAG pipeline, retrieved passages are prepended to the user query — the full sequence may be 8K–32K tokens
- Multi-head attention integrates information from multiple retrieved chunks simultaneously, without privileging any single chunk
- **Lost-in-the-middle problem:** Research shows certain attention heads disproportionately attend to the beginning and end of context — understanding MHA helps practitioners position critical information correctly
- **Anthropic's long-context Claude:** 200K tokens with GQA-style heads balance retrieval accuracy against KV-cache memory cost
- Multi-head diversity helps the model **synthesise conflicting sources** by allocating different heads to different retrieved documents



GenAI Application 5 --- Fine-Tuning with LoRA

Low-Rank Adaptation Targets MHA Weight Matrices

- LoRA (Hu et al. 2021) inserts low-rank matrices A , B as additive updates to the MHA projection weights: $W + BA$
- Typical LoRA targets: W^Q and W^V of each MHA layer — understanding MHA structure is essential to apply LoRA correctly
- Rank $r = 4$ to 64 reduces trainable parameters from billions to millions
- **QLoRA**: combines 4-bit quantisation with LoRA on MHA weight matrices to fine-tune 70B models on a single 48 GB GPU
- All major fine-tuning frameworks (HuggingFace PEFT, Axolotl, LLaMA-Factory) expose LoRA target modules as MHA layer names

Understanding which matrices form MHA is a practical prerequisite for running LoRA fine-tuning in production.



Common Misconceptions --- Part 1

Do Not Confuse These

- **"More heads always means better performance."**
Not always. Beyond a point, adding heads gives diminishing returns. Head pruning research (Michel et al. 2019) shows many heads can be removed with minimal quality loss. The optimal head count is a hyperparameter.
- **"Each head attends to a completely different set of tokens."**
Oversimplification. Heads often share overlapping attention patterns, especially in lower layers. Specialisation is partial, not absolute.
- **"The output projection W^O just reshapes dimensions."**
No. W^O is a fully learned linear transformation that mixes information across all heads — it is not a simple reshape or copy.



Common Misconceptions --- Part 2

More Things to Watch Out For

- **"GQA and MQA are only for inference efficiency — training is unaffected."**
Partly false. GQA reduces KV memory during training too, enabling larger batch sizes and longer sequences, which can improve training throughput.
- **"Cross-attention and self-attention are completely different mechanisms."**
No. They share the same scaled dot-product formula. The only difference is whether Q, K, V come from the same source or different sources.
- **"Head specialisation is manually designed by the model architects."**
No. Specialisation emerges entirely from training on data. No head is pre-assigned a role such as "syntactic head" or "semantic head".



Counting Parameters in a Multi-Head Attention Layer

Where the Parameters Live

- **Query projections:** $h \times d_{\text{model}} \times d_k = d_{\text{model}}^2$ parameters (across all heads)
- **Key projections:** d_{model}^2 parameters
- **Value projections:** d_{model}^2 parameters
- **Output projection W^O :** d_{model}^2 parameters
- **Total per MHA layer:** $4 \times d_{\text{model}}^2$
- For Llama 3 8B: $d_{\text{model}} = 4096$, MHA per layer = $4 \times 4096^2 \approx 67\text{M}$ parameters; with 32 layers $\approx 2.1\text{B}$ of 8B total reside in MHA blocks

MHA weight matrices are the primary targets of parameter-efficient fine-tuning because they contain $\sim 25\text{--}30\%$ of total model parameters.

Summary



Multi-Head Attention — Key Takeaways

1. A single attention head cannot capture all relationship types simultaneously; **multiple heads** solve this by learning independent subspace projections
2. MHA runs h scaled dot-product heads in parallel, then **concatenates and projects** their outputs via $\mathbf{W}^O$
3. Each head uses $d_k = d_{\text{model}}/h$, preserving total compute at the same cost as one full-dimension head
4. **GQA** (Llama 2/3, Mistral) reduces KV cache by sharing K/V heads across groups of query heads — now the standard in production LLMs
5. MHA weight matrices ($\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V, \mathbf{W}^O$) are the primary targets of LoRA fine-tuning
6. Cross-attention uses the same formula but draws Q from one sequence and K/V from another — enabling image-language and seq2seq models



What You Needed to Know Before This Session

Prerequisites in the Curriculum

- **Self-Attention Mechanism** — Scaled dot-product attention, Q/K/V matrices, softmax scoring, causal masking
- **What is a Transformer?** — High-level encoder-decoder structure
- **Autoregressive Generation** — Token-by-token decoding
- **Foundation Models Overview** — Why scale and architecture matter



What This Concept Unlocks

Immediate Next Concepts

- Positional Encoding (Module 2.4)
- Feed-Forward Networks in Transformers (Module 2.5)
- Decoder-Only Architecture (Module 2.6)
- Full Transformer Block Walkthrough (Module 2.7)

Downstream Concepts

- LoRA and QLoRA: Efficient Fine-Tuning (Module 7)
- Flash Attention and KV-Cache (Module 9)
- Cross-Attention in RAG and Diffusion (Module 5)
- Interpretability and Attention Visualisation (Module 8)

Next Steps



Recommended Actions

- Trace the MHA forward pass by hand for 2 heads on a 3-token sequence
- Visualise heads using BertViz or TransformerLens on GPT-2
- Read: Vaswani et al. 2017 — "Attention Is All You Need"
- Read: Ainslie et al. 2023 — "GQA: Training Generalised Multi-Query Transformer Models from Multi-Head Checkpoints"

Coming Up in Module 2

- Positional Encoding — injecting sequence order without recurrence
- Feed-Forward Networks — the other half of the Transformer block
- Layer Normalisation and Residual Connections
- Complete Transformer Block: putting it all together

Thank You

Multi-Head Attention | Module 2.2 | AI-TR-FN-TH-000003